

Distance Tracker Project

- **Purpose**

The purpose of this project is to display distance information for each angle in an intuitive way.

- **Features**

1. Device management

Users can select the device installed at the point where they want to measure distance.

2. Displaying distance at each angle

The Distance Tracker visualizes distance data for each angle, similar to radar equipment.

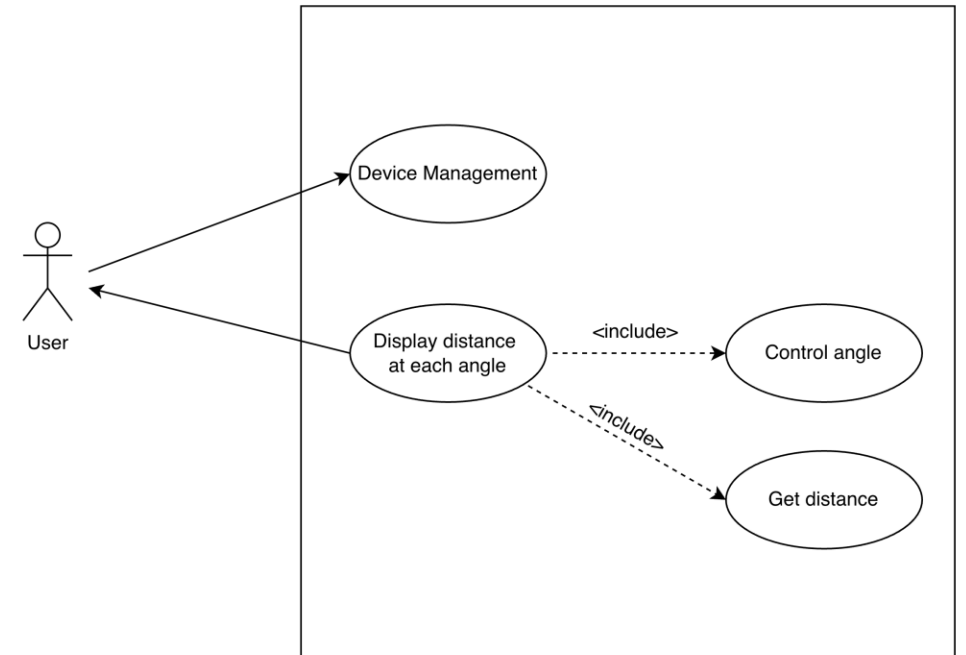


Figure1. Usecase Diagram

Sequence of program

1. Requesting distance at a specific angle

The user sends a command that includes the desired angle.

2. Parsing the Angle from the Command

The command is composed of characters that include the angle information.

The angle is parsed and converted into an integer value.

3. Rotating the servo motor to the requested angle

The device controls the servo motor to move to the specified angle. It waits until the servo motor reaches the target angle.

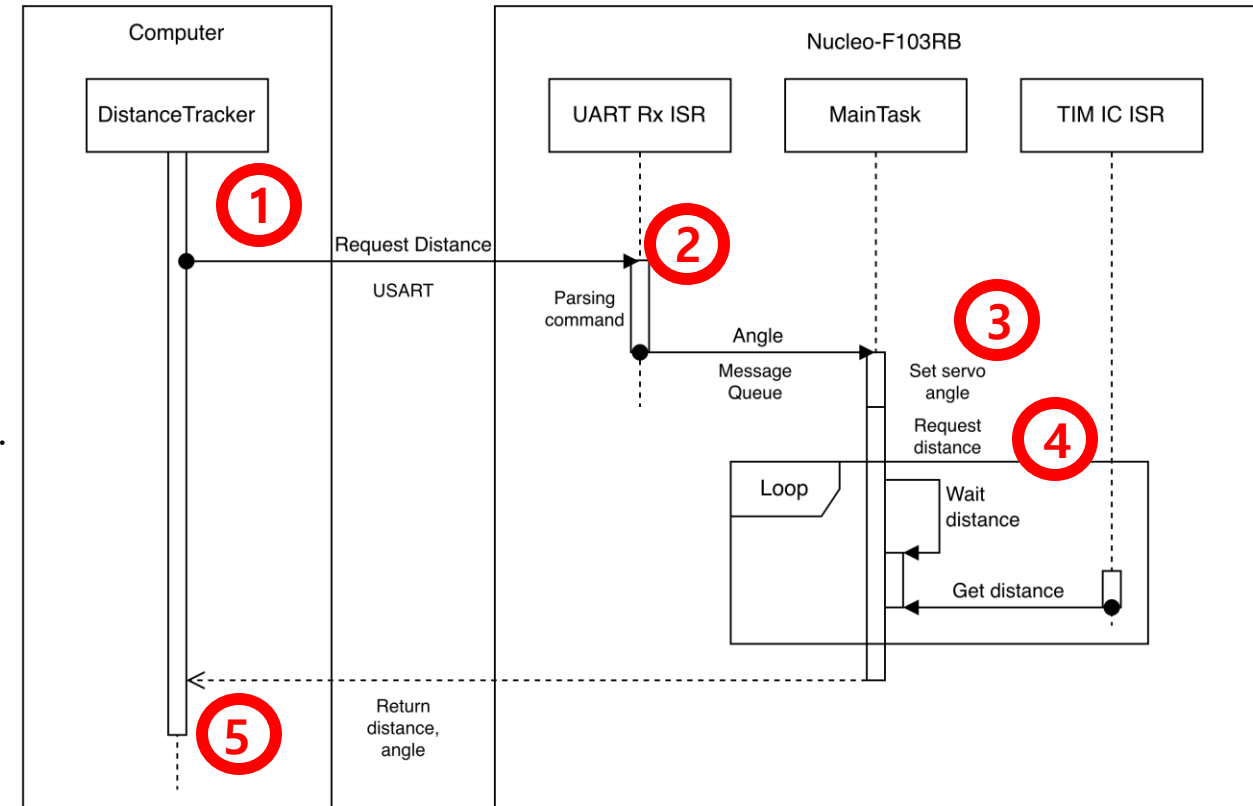
4. Acquiring distance using distance sensor

The device sends a trigger signal to the distance sensor to measure the distance.

It waits until the distance measurement is completed.

5. Returning the requested information to the user

The Distance Tracker obtains the distance at the requested angle. The measured distance is then displayed on the user interface.



UART

- **Operation Principle**

UART is a Serial Asynchronous Communication protocol that allows two devices to exchange data without sharing a common clock signal. It relies on a predefined Baud Rate to synchronize data sampling. Each byte is wrapped in a "frame" that uses Start and Stop bits to signal the beginning and end of a transmission.

- **Key Specification**

Communication Speed: 115200 bps

Word Length: 8bits (Parity None)

- **UART Standards**

Idle State: The line is held High (1) when no data is being sent.

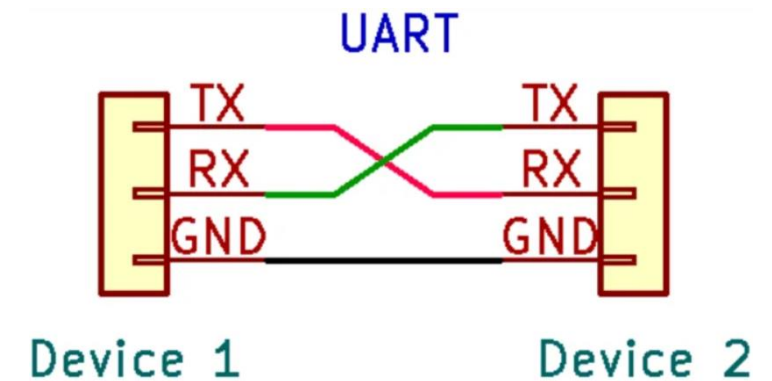
Start Bit: A transition from High to Low (0) for 1-bit duration to signal the start of a frame.

Data Bits: The actual data (5 to 9 bits, typically 8-bit LSB first).

Stop Bit: The line returns to High (1) for 1 or 2-bit durations to signal the end of a frame.

- **Wiring Pinout (Connection in STM32)**

Using USB-UART



Configuration

- **STX / ETX Settings (Debug Mode)**

For development and monitoring convenience, the protocol utilizes printable ASCII characters instead of standard control codes.

STX (Start of Text): Assigned to '@' (\$0x40\$).

ETX (End of Text): Assigned to '!' (\$0x21\$).

- **Device Settings**

Using USART2 device.

- **Payload Specifications**

The message frame follows a fixed-length format to ensure consistent parsing.

Buffer: Once the angle is successfully parsed, the resulting data is pushed into the designated AngleQueueHandle.

Structure: Total payload length is **5 characters** (excluding delimiters).

Format: Consists of one character identifier followed by three digits that are castable to an integer (e.g., @S123!).

Use Case: MainTaskHandler retrieves uint16_t angle data from the queue (osMessageQueueGet) for servomotor positioning.

Configuration cont'd

- **DMA & Interrupt Configuration**

To maximize CPU efficiency and ensure real-time responsiveness, the hardware is configured as follows:

DMA Settings: All incoming UART data is directly loaded into the memory buffer via **Direct Memory Access (DMA)**, bypassing the CPU during transfer.

Interrupt Settings: The system is configured to trigger interrupts on **IDLE (Line Idle Detection)** and **TC (Transfer Complete)** events.

- **Parser Architecture**

Implementation: The code within the ISR callback processes the DMA buffer immediately upon the detection of an IDLE line condition.

Parser Callback Specification

- **Detailed Callback Specification**

Execution Flow: The callback is triggered upon the occurrence of TC (Transfer Complete) and IDLE (Idle Line Detection) interrupts. Once invoked, the callback function executes `Handle_UART_Receive_IDLE()` to process parsing.

- **Callback Function**

Primary Buffering: Incoming data is initially captured into a circular `rxBuffer` managed by the UART peripheral. This ensures that data is safely stored in memory without CPU intervention during the transfer process.

Event Handling: The `HAL_UARTEx_RxEventCallback` serves as the primary entry point for processing. It is triggered by the hardware upon an IDLE line detection or a Transfer Complete (TC) event, subsequently invoking the `Handle_UART_Receive_IDLE()` handler.

Validation & Parsing: The handler orchestrates specialized sub-functions, most notably `process_incoming_servo_data()`, to verify frame integrity. This parser enforces strict protocol rules:

Delimiters: It validates the presence of both STX (@) and ETX (!) markers.

Payload Length: It confirms that the payload length is exactly 5 characters.

Error Handling: Any data frame that fails to meet these criteria is immediately discarded.

Note: Detailed functional descriptions for the parser are currently omitted, as the specific input and output communication requirements are subject to frequent changes during the ongoing development process.

SG90 (Servo Motor)

- **Operation Principle**

The SG90 is an actuator controlled by a Pulse Width Modulation (PWM) signal to set specific output shaft angles.

- **Key Specification**

Operating speed : 0.1s / 60 degree

Operating voltage : 4.8V (~5V)

Frequency : 50Hz (20ms Period)

- **PWM Signal Standards**

0 degree : 1.5ms High Pulse

90 degree : 2ms High Pulse

-90 degree : 1ms High Pulse

- **Wiring Pinout (Connection in STM32)**

Orange : Signal / PWM Control Input / TIMx_CHx Pin

Red : VCC / Power (+5V) / 5V Pin

Brown : GND / Ground (0V) / GND Pin



STM32 Timer (TIM) Configuration

- **Counter Settings**

To achieve a 20ms period, the Prescaler and ARR (Auto-Reload Register) must be calculated based on the Clock frequency.

- **Calculation (Example for 64MHz Clock)**

Prescaler: Set to 639 (Reduces clock to 100kHz).

Counter Period (ARR): 1999 (Sets frequency to 50Hz, or 20ms).

- **Channel Configuration**

Enable "PWM Generation CHx" in the STM32CubeIDE Device Configuration Tool.

- **HAL Integration**

The class relies on “**stm32f1xx_hal.h**” to interface with the MCU hardware

Class Architecture

Detailed Class Specification : class SG90

The SG90 class provides a high-level interface for controlling a standard SG90 servo motor using the STM32 HAL library

Private Member Attributes

TIM_HandleTypeDef* _htim
Pointer to the hardware timer instance used for PWM generation
uint32_t _channel
The specific hardware timer channel assigned to this servo
int _currentAngle
Internal state tracking of the current servo position to compute movement deltas

Class Architecture

Public Member Functions

SG90(TIM_HandleTypeDef* htim, uint32_t channel)

- **Description** : Constructor that initializes the servo object
- **Parameters**
 - **htim** : Pointer to a TIM_HandleTypeDef structure that contains the configuration information for the specified Timer
 - **channel** : Timer channel to be used
- **Note** : Automatically calls **HAL_TIM_PWM_Start()** during initialization

uint32_t setAngle(int angle)

- **Description** : Sets the servo motor to a specific angle
- **Parameters**
 - **angle** : Target angle in degrees (Input range : 0 to 180)
- **Returns** : uint32_t (The estimated time in milliseconds required for the motor to reach the target angle)

Implementation Logic

Safety Constraints

The input angle is clamped between 0 degree and 180 degree to prevent mechanical stress or software overflows

Pulse Width Formula

$$\text{pulse_val} = 50 + (\text{angle} * 200 / 180)$$

This maps the degree input to a compare value that corresponds to the required 1ms-2ms pulse width

Dynamic Delay Calculation

$$\text{delay_ms} = (\text{abs}(\text{angle} - \text{_currentAngle}) * 400) / 180$$

By calculating the difference between the current and target angles, the function returns a millisecond value for software-side synchronization

HC_SR40 (Ultrasonic Sensor)

- **Operation Principle**

Measures distance by emitting a 40kHz ultrasonic pulse and calculating the time it takes for the echo to return

- **Key Specification**

Operating voltage : 5V DC

Measurement Range : 2~400cm

Effectual Angle : < 15 degree

Trigger Input Pulse width : 10uS

- **Wiring Pinout (Connection in STM32)**

VCC : 5V Power Supply

Trig : Trigger Input (Connected to STM32 GPIO Output)

Echo : Echo Output (Connected to STM32 Timer Input Capture Pin)

GND : Ground



STM32 Input Capture (IC) Configuration

- **Counter Settings**

Timer Input Capture (IC) : Used to precisely measure the duration of the High pulse on the Echo pin.

- **Calculation (Example for 64MHz Clock)**

Prescaler: Set to 63

Result : Each timer “tick” represents exactly 1uS

- **Interrupt Management**

TIM_IC_Start_IT : Enables Input Capture with interrupts to detect both Rising and Falling edges.

- **Distance Formula**

Since Sound Speed is approx. 340m/s, the formula used in the code is : **Distance = Time/58**

Class Architecture

Detailed Class Specification : class HC_SR04

The HC_SR04 class manages the triggering and timing analysis for the ultrasonic sensor using Timer Input Capture.

Public Member Attributes

TIM_HandleTypeDef* const _htim
Constant pointer to the allocated hardware timer instance
GPIO_TypeDef* const _triggerPort / const uint16_t _triggerPin
Hardware address and pin number for the ultrasonic trigger signal
bool _isDone
Status flag indicating the measurement cycle is finished
int32_t _distance
Internal buffer storing the most recent distance result

Class Architecture

Public Member Functions

HC_SR04(const TIM_HandleTypeDef* timer, const GPIO_TypeDef* port, const uint16_t pin)

- **Description** : Constructor that initializes the sensor instance and maps hardware resources
- **Functionally** : Assigns the timer handle and GPIO pins, then initiates the Input Capture interrupt mode
- **Note** : Input Capture interrupt mode in STM32 is “**HAL_TIM_IC_Start_IT**”

void setTrigger()

- **Description** : Triggers the ultrasonic sensor to emit sound waves
- **Functionality** : Resets internal flags, sets the Trigger pin High for 1ms, and then resets it to Low

bool isDone() const

- **Description** : Checks the completion status of the distance measurement
- **Return** : **true** if the echo pulse has been fully captured and processed, **false** otherwise

int32_t getDistance() const

- **Description** : Retrieves the finalized distance calculation
- **Return** : The measured distance in centimeters. Returns **-1** if the measurement is incomplete

Implementation Logic

Triggering Mechanism

Resets the state (`_isDone = false`) and toggles the Trigger pin for a 1ms duration using **HAL_Delay(1)**

Input Capture State Machine

- Rising Edge : Captures the start time and switches the polarity to detect the Falling Edge
- Falling Edge : Captures the end time and switches back to Rising Edge detection for the next cycle

Interval Calculation

Handles timer overflow : $\text{Difference} = (p2 \geq p1) ? (p2 - p1) : ((ARR - p1) + p2)$